1) Cyber-physical systems (CPS), such as automated vehicles, robotics, medical equipment, manufacturing equipment, IoT, combine software, physical hardware, communication networks, and real-world processes. The development of these systems increasingly involves continuous software engineering and DevOps practices, including frequent integration, automated testing, continuous monitoring, and collaboration between development and operations teams. However, applying these practices to CPS is more difficult than applying them to purely software-based systems because software changes must operate correctly under physical and hardware constraints.

My doctoral research investigates data-driven development in cyber-physical systems. Data-driven development refers to using data generated during software development and system operation to support engineering decisions. In a DevOps context, relevant data may include software changes, build and integration results, test results, deployment information, defect reports, and system or hardware logs. Analysing such data can help engineering teams identify patterns, anticipate failures, allocate resources, and decide which tests should be prioritised.

A central challenge is that CPS development depends on hardware environments that may be expensive, scarce, slow to configure, or difficult to reproduce. A software component may perform correctly in a simulated or isolated environment but behave differently when integrated with sensors, control units, networks, and other physical components. Furthermore, modern CPS may contain both rule-based software, whose behaviour is explicitly defined in code, and AI-enabled components, whose behaviour is learned from data. This combination introduces additional concerns related to testing, uncertainty, system integration, traceability, resilience and reliability.

My research has two connected objectives. The first is to develop a deeper empirical understanding of how data-driven development is applied in industrial CPS development. This includes studying current practices, challenges, and needs through industrial case studies and an investigation of the existing research literature. The second objective is to iteratively develop and evaluate tools that use data analytics, machine learning, or artificial intelligence to support development decisions. Potential applications include test planning, failure prediction, resource allocation, and the prioritisation of engineering activities.

The research follows an empirical software-engineering and iterative design-research approach in collaboration with industrial partners like Volvocars, Bosch, Tetrapak, etc through NextG2com competence centre from Lund University. This makes it possible to study real development environments and evaluate whether proposed tools provide practical value. The topic is closely related to AI Engineering because useful analytical or machine-learning models cannot be considered in isolation. They must be integrated into reliable software processes, evaluated under realistic hardware conditions, and provide trustworthy information to the engineers responsible for decisions.

2) 1st topic : Verification and validation in data-driven CPS development (Lecture 2)

Verification asks whether a system has been built correctly according to its specification, whereas validation sees whether the right system has been built for its intended purpose. Both are important in cyber-physical systems, but validation is particularly challenging because the behaviour of the software depends on its interaction with hardware, networks, physical processes, and the operating environment. This distinction relates directly to my research on data-driven development in cyber-physical systems. A data-analytics or machine-learning tool may be verified by checking whether its implementation is correct, its data pipeline functions as intended, and its outputs satisfy defined technical requirements. However, these checks do not establish whether the tool provides useful support for industrial engineering decisions. For example, a model for prioritising tests may achieve good predictive

performance on historical DevOps data but still be unsuitable in practice if it recommends tests that require unavailable hardware, ignores safety critical system configurations, or does not provide its results early enough in the development process.

2nd topic : Behavioral software engineering and human decision-making (Lecture 3) Decisions are influenced by experience, cognitive biases, organisational priorities, time pressure, group dynamics, and the specific context in which the decision is made.

This principle is highly relevant to my research because data-driven development ultimately aims to support decisions made by people. A decision-support tool might analyse DevOps data and recommend particular tests, resources, or engineering actions. However, its usefulness depends on whether engineers understand, trust, and act upon its recommendations. Engineers may reject a technically accurate recommendation if its reasoning is unclear or if it conflicts with their experience. Conversely, they may trust an automated recommendation too strongly and fail to consider limitations in its data or model.

In CPS development, these behavioural issues may become more significant because engineers from different disciplines must collaborate. Software developers, hardware engineers, test engineers, data scientists, and managers may have different priorities and interpretations of risk. Limited hardware resources can also create competition between teams and influence decisions about which components or configurations receive testing time.

3) 1st topic (Julian-guest lecture) : Adapting requirements engineering for AI-enabled systems

From Julian Frattini's guest lecture is that requirements engineering for AI-enabled systems cannot simply reuse all assumptions made for traditional software. In traditional software, behaviour is largely defined explicitly through source code. In an AI-enabled component, part of the behaviour is learned from data. Its behaviour therefore depends not only on its implementation but also on the training data, model, operating context, and changes occurring after deployment. Consequently, qualities such as accuracy, reliability, fairness, maintainability, and retrainability may require different definitions and priorities. The guest lecture distinguished between the context, requirements, and system levels by considering why a system is needed, what it must achieve, and how it will be implemented.

This principle is relevant to my research because modern cyber-physical systems may combine rule-based and data-driven components. These components interact with hardware, networks, sensors, and physical processes. Requirements must therefore cover more than the functional behaviour of individual software components. They must also address data quality, model performance under different operating conditions, hardware compatibility, uncertainty, monitoring, and the conditions under which a model should be retrained or replaced.

2nd topic (Lenberg guest lecture) : AI-mediated workflows, changing roles, and accountability

Lenberg's guest lecture presented software engineering as more than executable code. Development also includes requirements, task descriptions, human workflows, handovers, quality-assurance gates, monitoring, decision rights, and professional responsibilities. When AI is introduced, these elements become part of an AI-mediated workflow involving generated artifacts, automated evaluations, confidence thresholds, and human-agent interactions. As a result, AI changes both the work performed and the allocation of responsibility.

This principle relates strongly to my research because a data-driven decision-support tool would not operate independently of an organisation. For example, a tool might recommend which tests should be

executed when hardware resources are limited. However, the practical effect of the recommendation depends on who reviews it, who may override it, what confidence level is acceptable, and who remains responsible if a relevant test is omitted. These questions are particularly important in industrial CPS, where incorrect decisions can affect reliability or safety.

Introducing an AI-supported tool may also expose weaknesses in an existing DevOps process. An organisation may discover that responsibility for test selection is unclear, that teams use inconsistent escalation procedures, or that important knowledge exists only informally among experienced engineers. Automation does not automatically resolve these problems. Instead, unclear workflows and decision rights can be transferred into the automated process or become more visible when the AI tool is deployed.

Therefore, the design of my research tools should explicitly include human oversight and accountability.

4) Differences between the roles: The chapters describe data scientists as mainly focused on data, statistical methods, model development, and experimental evaluation. They often measure success using model accuracy. Software engineers focus on complete products, including requirements, architecture, testing, deployment, security, scalability, maintenance, cost, and user needs.

Do I agree? I generally agree, although the distinction is simplified. Building an accurate model and operating a reliable product require different expertise. However, the roles increasingly overlap. Data scientists need knowledge of deployment, monitoring, reproducibility, and software quality, while software engineers need to understand data quality, model uncertainty, drift, and prediction errors.

Application to my research: The system-wide perspective is especially relevant to my research on data-driven development in cyber-physical systems. A failure-prediction or test-prioritisation model is only one component within a larger environment containing DevOps pipelines, test infrastructure, engineers, and physical hardware. A highly accurate model may still be unsuitable if it recommends tests requiring unavailable hardware, produces results too late, or cannot explain its recommendations. Its value must therefore be judged by whether it improves engineering decisions and overall system quality.

Future of the roles: Data scientists and software engineers will probably remain distinct specialists, but both will need broader, broad expertise. Effective collaboration requires data scientists to understand engineering practices and software engineers to develop greater ML literacy.

Position of AI Engineering: I see AI Engineering as a combination of software engineering, data science, and MLOps, while also being partly distinct. MLOps supports model training, deployment, versioning, and monitoring. AI Engineering has a broader responsibility that also includes requirements, architecture, testing, safety, security, human-AI interaction, and accountability.

Effects of emerging AI technologies: Foundation models, RAG systems, prompt pipelines, coding assistants, and agentic workflows further blur professional boundaries. Although teams may not train the underlying models, they must still manage data, integration, evaluation, testing, monitoring, permissions, and errors.

5) Paper- 1 What About the Data? A Mapping Study on Data Engineering for AI Systems

(Petra Heck. 2024. What About the Data? A Mapping Study on Data Engineering for AI Systems. In Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering - Software Engineering

for AI (CAIN '24). Association for Computing Machinery, New York, NY, USA, 43–52. https://doi.org/10.1145/3644815.3644954)

Core ideas and software-engineering importance

Heck's CAIN 2024 paper argues that although data is fundamental to AI systems, data engineering receives less attention than model development. The paper maps 25 peer-reviewed studies and examines data-engineering activities, tools, architectures, and lessons. It shows that data engineering includes the ingestion, transformation, storage, validation, versioning, and delivery of data. These activities must continue after deployment because production data must be monitored and prepared for future model updates.

An important finding is that most studies focus on individual training or production pipelines, while comprehensive guidance on organisational data architectures remains limited. The paper also identifies gaps concerning production-data monitoring, DataOps, data quality, and domain-specific data engineering. These findings are important to software engineering because an accurate model cannot remain reliable if the underlying data pipeline is incomplete, inconsistent, or difficult to reproduce. Data infrastructure should therefore be treated as a maintained software system rather than temporary preprocessing code.

Relation to my research

The paper relates directly to my research on data-driven development in cyber-physical systems. CPS organisations generate heterogeneous data, including software changes, build results, test outcomes, hardware configurations, system logs, and sensor measurements. These sources may use different formats and be owned by different teams. Before such data can support failure prediction or test prioritisation, it must be collected, connected, cleaned, and given sufficient context.

The paper also mentions automotive and IoT as important domains for specialised data engineering. In these environments, a failed test result may be difficult to interpret without knowing the software version, hardware configuration, test environment, and sensor conditions. Data quality and traceability are therefore essential parts of the decision-support problem.

Integration into a larger AI-intensive project

A possible larger project would be an AI-assisted test-planning platform for automotive CPS development. The platform would collect data from CI/CD pipelines, source-code repositories, test-management systems, hardware test rigs, and operational logs. An ML model would estimate the risk associated with each software change and recommend tests that should be executed on limited hardware resources.

The paper's ideas could be applied by creating a DataOps pipeline with automated ingestion, validation, versioning, transformation, and monitoring. The platform would record the origin of each data item and connect model recommendations to the relevant code, tests, and hardware configurations. My WASP research could contribute empirical studies of industrial data practices and the design and evaluation of the test-planning component.

Adaptation of my research

To better support this idea, I would expand my research from analysing available DevOps data to explicitly studying how that data is produced and maintained. I could investigate data-quality problems, missing links between software and hardware artifacts, and organisational responsibility for data. A

prototype could include data validation, lineage, and pipeline monitoring alongside the prediction model. This would allow the evaluation to consider not only model accuracy but also reproducibility, data reliability, and practical decision value.

Paper-II Foundation Models for Software Engineering of Cyber-Physical Systems: The Road Ahead. (Chengjie Lu, Pablo Valle, Jiahui Wu, Erblin Isaku, Hassan Sartaj, Aitor Arrieta, and Shaukat Ali. 2026. Foundation Models for Software Engineering of Cyber-Physical Systems: The Road Ahead. ACM Trans. Softw. Eng. Methodol. 35, 9, Article 278 (September 2026), 58 pages. https://doi.org/10.1145/3830403)

Core ideas and software-engineering importance

Lu et al. present a research roadmap for using foundation models throughout CPS software engineering. They consider requirements engineering, system modelling, software development, testing, debugging, repair, and evolution. Their main argument is that CPS development may benefit not only from language models but also from multimodal foundation models capable of processing text, code, images, audio, sensor signals, and other CPS data.

For testing, the authors propose using foundation models for test generation, selection, minimisation, and prioritisation. However, they note that foundation models have limitations in numerical reasoning and may generate incorrect information. They therefore recommend combining them with established techniques such as search-based testing and numerical solvers. They also identify hallucinations, uncertainty, confidentiality, and verification of generated artifacts as important open challenges. These issues matter because incorrect requirements, code, or tests could have physical consequences in a CPS.

Relation to my research

This paper closely matches my research area because it focuses specifically on software engineering for CPS. My project studies how data analytics and AI can support engineering decisions under hardware and resource constraints. Foundation models could analyse multiple development artifacts together, such as requirements, code changes, test descriptions, logs, and hardware information. This could provide richer decision support than a model trained on only one structured dataset.

The paper's discussion of test prioritisation is especially relevant. CPS regression suites can be large and expensive because some tests require simulators, hardware-in-the-loop environments, or physical devices. A foundation model could help interpret a software change and identify relevant tests, but its recommendation would still need validation against historical evidence and system constraints.

Integration into a larger AI-intensive project

The automotive test-planning platform described above could include a foundation-model assistant. When a code change is submitted, the assistant could analyse the change, related requirements, previous failures, test descriptions, and hardware configurations. It could propose relevant tests and explain why each test is recommended. A conventional prediction model could estimate failure risk, while optimisation or search-based methods could select a feasible test set based on hardware availability and execution time.

This hybrid design would use the foundation model for interpreting heterogeneous artifacts and traditional methods for numerical optimisation and constraint handling. Engineers would review the final recommendations, particularly for safety-critical changes. My research could contribute the industrial data, decision-support design, and empirical evaluation needed to determine whether the approach improves test planning.

Adaptation of my research

I could adapt my project by adding a research question about whether foundation models improve data-driven test selection in CPS. The evaluation should compare an FM-assisted approach with existing rule-based and statistical baselines. Relevant outcomes would include fault detection, hardware cost, execution time, explanation quality, and engineer trust.

The research would also need safeguards. Recommendations should be traceable to evidence, checked against hardware constraints, and approved by an engineer when safety is involved. Since industrial code and logs may be confidential, locally deployable models or controlled retrieval-augmented generation could be considered. This extension would preserve the empirical and industrial focus of my doctoral project while addressing emerging AI-engineering challenges.

Evidence, originality, and GenAI-use transparency

References:

1) Robert Feldt's Lectures 2 and 3

2) Julian Frattini's and Per Lenberg's guest lecture

3) The two CMU book chapters

4) Paper -1 and Paper-2 reference in answer 5.

5) Pushpa Priyanka Palesetti, Emelie Engström, Emma Söderberg, Al-Hussein Hameed Jasim, Andreas Bexell, Nikolaos Korkakakis, Amir Aminifar, Robert Lagerstedt, Pontus Olsson, Per Sigurdson, Markus Borg, Nadim Hagatulah, Alma Orucevic-Alagic, and Maria Kihl. 2025. Towards Sustainable DevOps for Cyber Physical Systems. In Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion '25). Association for Computing Machinery, New York, NY, USA, 1437–1440. https://doi.org/10.1145/3696630.3728708

6) Palesetti, Pushpa Priyanka and Engström, Emelie and Söderberg, Emma and Orucevic-Alagic, Alma, Understanding Drift in DevOps for Cyber Physical Systems: A Multi-industry Mixed Method Study (Registered Report) (June 27, 2026). Available at SSRN: https://ssrn.com/abstract=7007719 or http://dx.doi.org/10.2139/ssrn.7007719

I place the greatest trust in the peer-reviewed publications (paper-1 &2 from answer 5) and reference 5 and academic book-CMU because their arguments and evidence are documented and have undergone academic review.

I used ChatGPT to help interpret the assignment instructions, brainstorm connections between the course material and my research, organise the essay, and improve language clarity. I read the original sources, checked the citations and factual claims, and revised the suggestions to reflect my own research context and understanding.